GUVI / HCL | FINAL PROJECT 04

Three-Tier Blog Application
Execution Evidence

Application • Docker • Jenkins • SonarQube • Trivy • Kubernetes

GITHUB REPOSITORY

https://github.com/naviktg/Guvi-Project-4

EXECUTION ENVIRONMENT

Windows + Docker Desktop Kubernetes + Jenkins + Docker Hub

Prepared from screenshots and pipeline output supplied in the project conversation.

Prepared: 02 November 2026

Evidence summary

What the included screenshots and supplied console output demonstrate

- Application home page and a published post are shown; the learner confirmed sign-in, post creation, comments and refresh/persistence checks worked.
- Docker images for the backend and frontend were built. Docker Compose output shows MySQL and backend healthy, with the frontend started.
- Jenkins Build #20 is shown as successful. Its page displays SonarQube Quality Gate Passed and archived Trivy JSON report artifacts.
- The supplied Jenkins console output records Docker Hub pushes and successful backend/frontend Kubernetes rollouts.
- The final Jenkins console screenshot shows Kubernetes deployment completion, service information and Finished: SUCCESS.

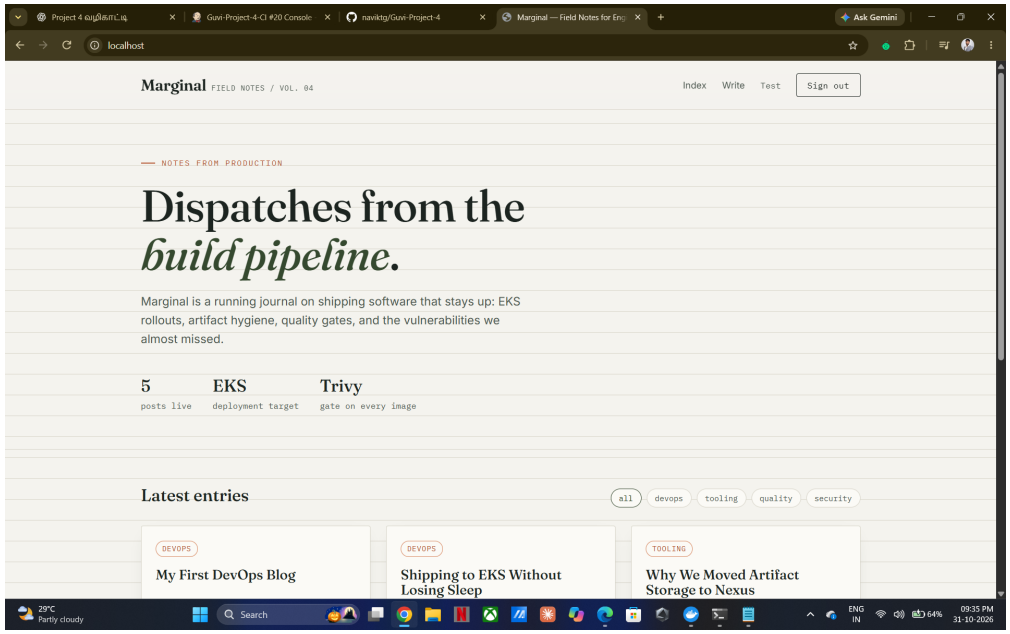
IMPORTANT IMPLEMENTATION SCOPE NOTE

The assignment brief specifies Amazon EKS and allows Nexus or ECR as the image repository. The evidence in this pack documents deployment to local Docker Desktop Kubernetes, not Amazon EKS.

The pipeline pushed images to Docker Hub, not Nexus/ECR. Prometheus/Grafana dashboards and centralized logging are not evidenced in the supplied materials. This report does not claim those items were completed.

Application validation

Screenshots of the working blog interface and a published post



A. Application home page - signed-in navigation and latest entries

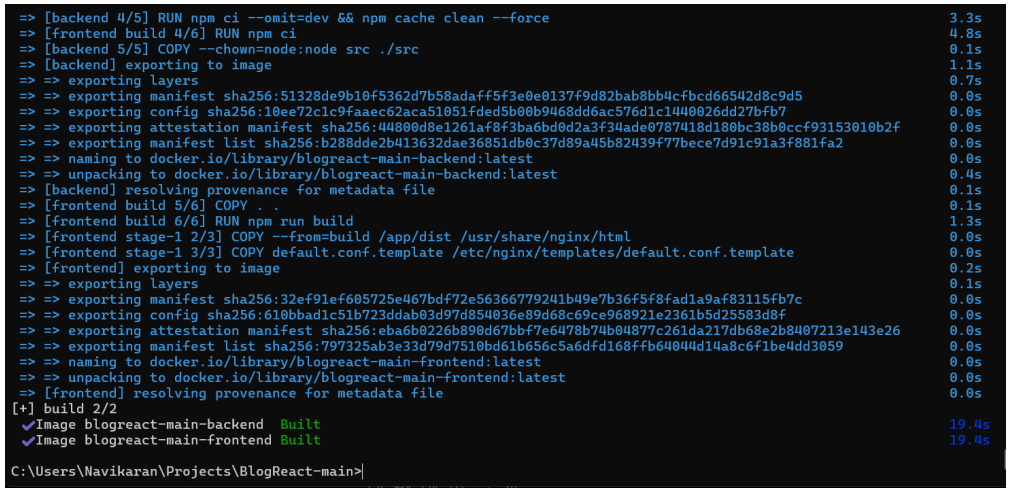


B. Published post detail with user note/comment area

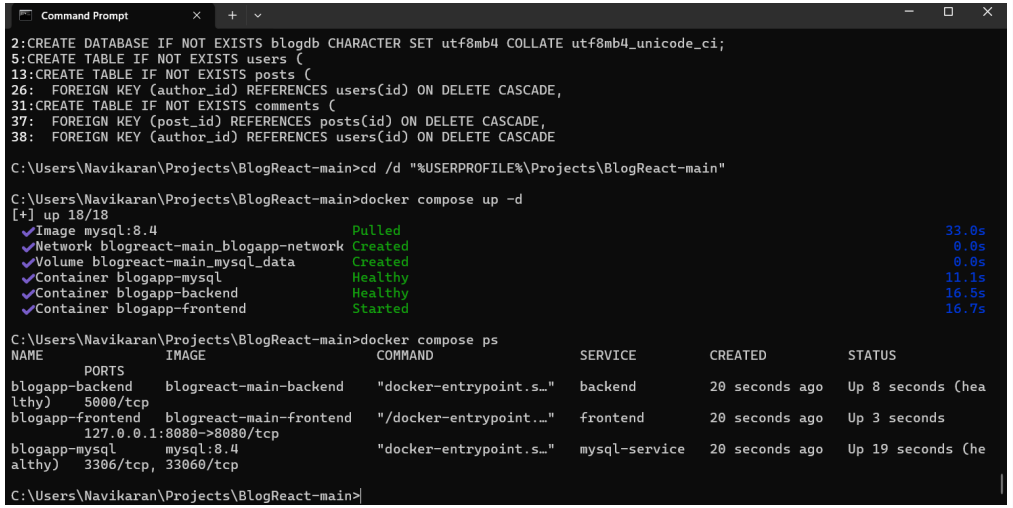
The learner reported that registration/login, creating posts, adding comments and refreshing the page to confirm the content remained available were all tested successfully. The screenshots provide visual evidence of the application UI; the persistence check is learner-reported.

Container build and local runtime

Build output and Docker Compose health status from the shared terminal screenshots



A. Docker build - backend and frontend images built

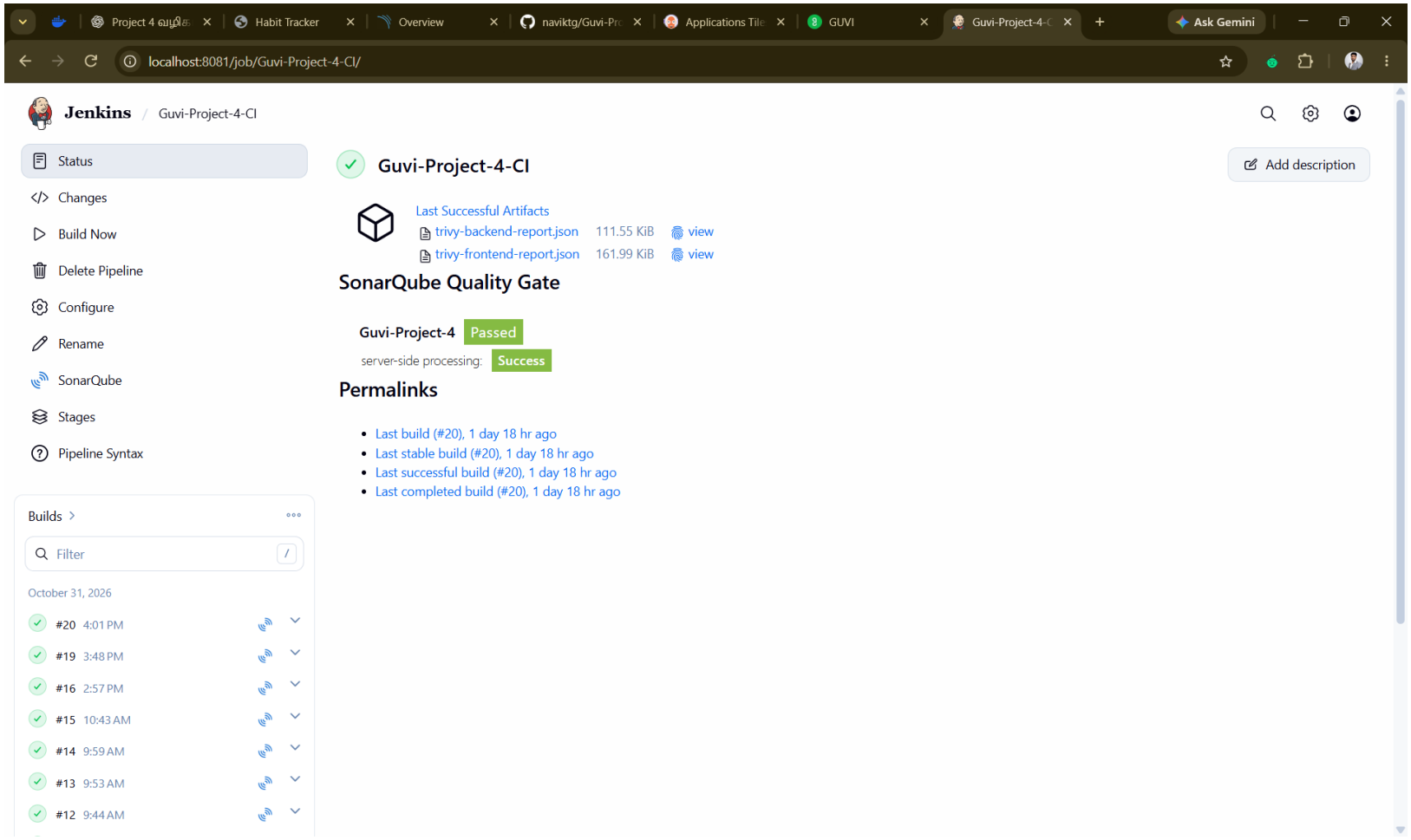


B. Docker Compose - MySQL and backend healthy; frontend started

These screenshots document local container image creation and the initial three-service Compose run. The later Kubernetes deployment is documented in the Jenkins evidence pages.

Jenkins Build #20

Job status, latest successful build reference, Quality Gate result and archived scan reports



Jenkins job page - Build #20, SonarQube Quality Gate Passed, Trivy JSON artifacts

Visible archived artifacts: trivy-backend-report.json and trivy-frontend-report.json. The page also shows SonarQube Quality Gate status as Passed. The archived reports should be opened in Jenkins when detailed CVE findings are required; the screenshot alone does not establish that every vulnerability was remediated.

Image publishing and Kubernetes rollout

Key excerpts from the Jenkins console output shared in the conversation

DOCKER HUB PUSH

Console output reported that both images were pushed successfully.

Final confirmation line:

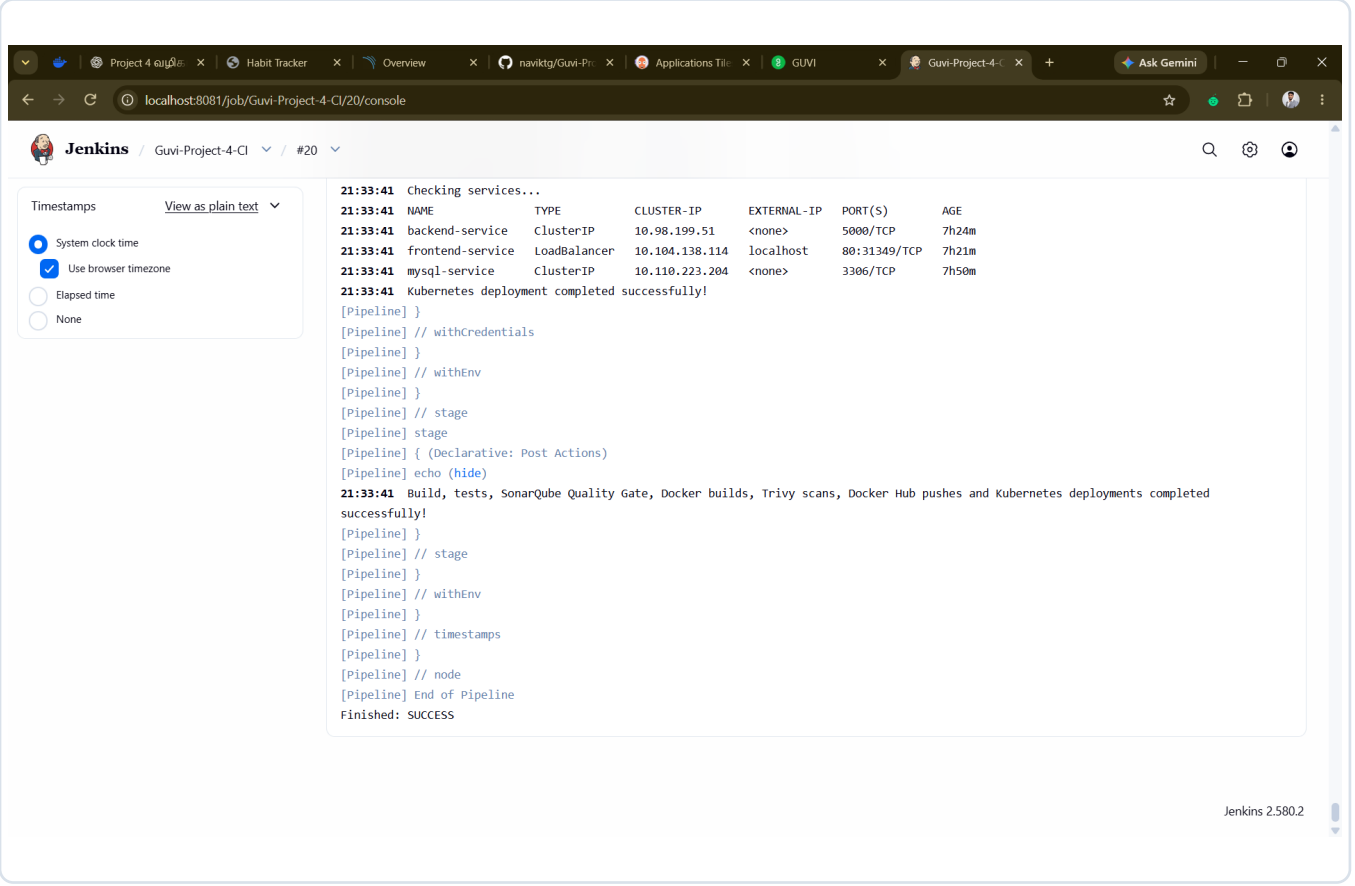
Both images pushed successfully to Docker Hub!

KUBERNETES ROLLOUT

Backend deployment: successfully rolled out.

Frontend deployment: successfully rolled out.

Deployment summary: backend 2/2, frontend 2/2, MySQL 1/1.

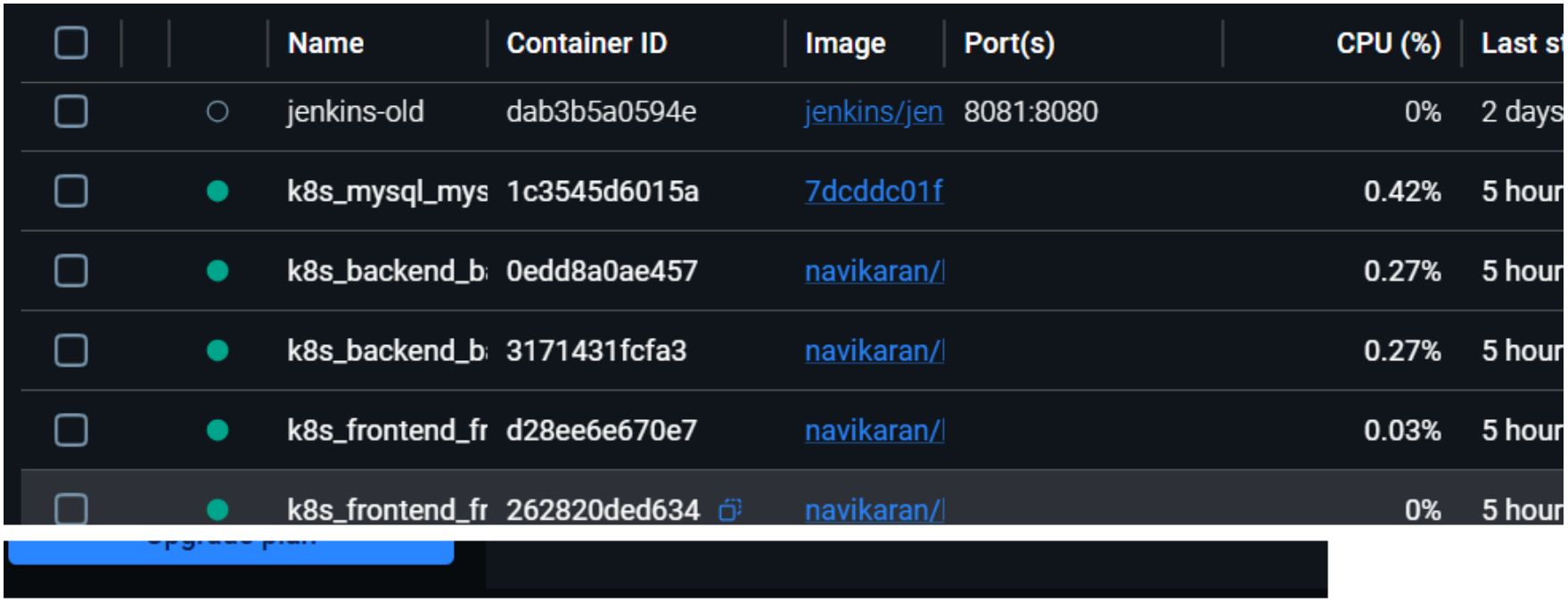


Build #20 console tail - services listed and Finished: SUCCESS

The push and rollout statements above are excerpts from the learner-pasted Jenkins console output, not a separate screenshot. The screenshot at right independently shows the final deployment-completed message and pipeline success.

Docker Desktop workload evidence

Relevant MySQL, backend and frontend Kubernetes containers from the supplied Docker Desktop screenshot



Name	Container ID	Image	Port(s)	CPU (%)	Last st
jenkins-old	dab3b5a0594e	jenkins/jen	8081:8080	0%	2 days
k8s_mysql_mys	1c3545d6015a	7dcddc01f		0.42%	5 hour
k8s_backend_b	0edd8a0ae457	navikaran/		0.27%	5 hour
k8s_backend_b	3171431fcfa3	navikaran/		0.27%	5 hour
k8s_frontend_fr	d28ee6e670e7	navikaran/		0.03%	5 hour
k8s_frontend_fr	262820ded634	navikaran/		0%	5 hour

Cropped view of Project 4 workload rows and Docker Desktop engine/Kubernetes status

This is local Docker Desktop Kubernetes evidence. It should not be labelled as an Amazon EKS cluster screenshot.

Evidence checklist and reviewer notes

Use this document as the execution-screenshot attachment alongside the GitHub repository link

EVIDENCE INCLUDED

- Application home and published post screenshots
- Docker backend/frontend build screenshot
- Docker Compose service health screenshot
- Jenkins Build #20 success / SonarQube gate / Trivy report links
- Jenkins deployment success console screenshot
- Docker Hub push and Kubernetes rollout console excerpts
- Local Docker Desktop Kubernetes workload screenshot

NOT PROVEN BY THIS EVIDENCE PACK

- Amazon EKS deployment and AWS LoadBalancer/Ingress URL
- Nexus or Amazon ECR repository push
- Prometheus/Grafana monitoring dashboards
- Centralized logging configuration
- Full Trivy finding remediation or a clean-vulnerability claim
- Database data retention after pod recreation

Repository: https://github.com/naviktg/Guvi-Project-4

Suggested GUVI submission: paste the GitHub repository URL in the GitHub URL field and share this PDF through a cloud document link in the Document URL field. Confirm the document link is accessible to the reviewer.

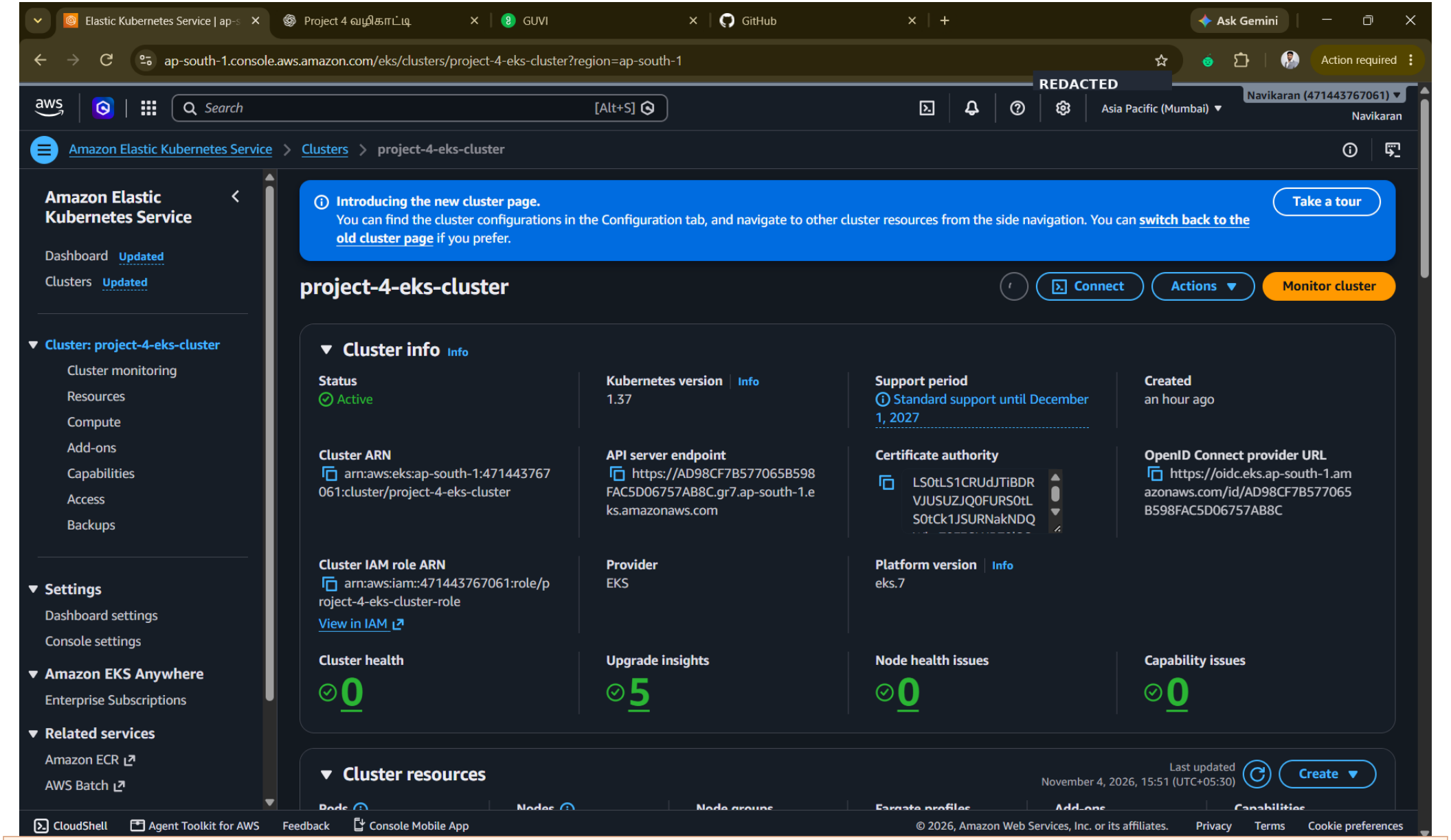
This report was compiled from screenshots and console output that were already shared in the conversation. It intentionally excludes screenshots containing exposed credentials or unrelated AWS/account details.

Amazon EKS Cluster – AWS Console Evidence

GUVI Project 04 | Screenshot captured 04 November 2026

What this screenshot confirms

The AWS EKS console displays **project-4-eks-cluster** with status **Active**, Kubernetes version **1.37**, provider **EKS**, and region **ap-south-1 (Mumbai)**. The console shows cluster health 0 and node health issues 0 at the time of capture.

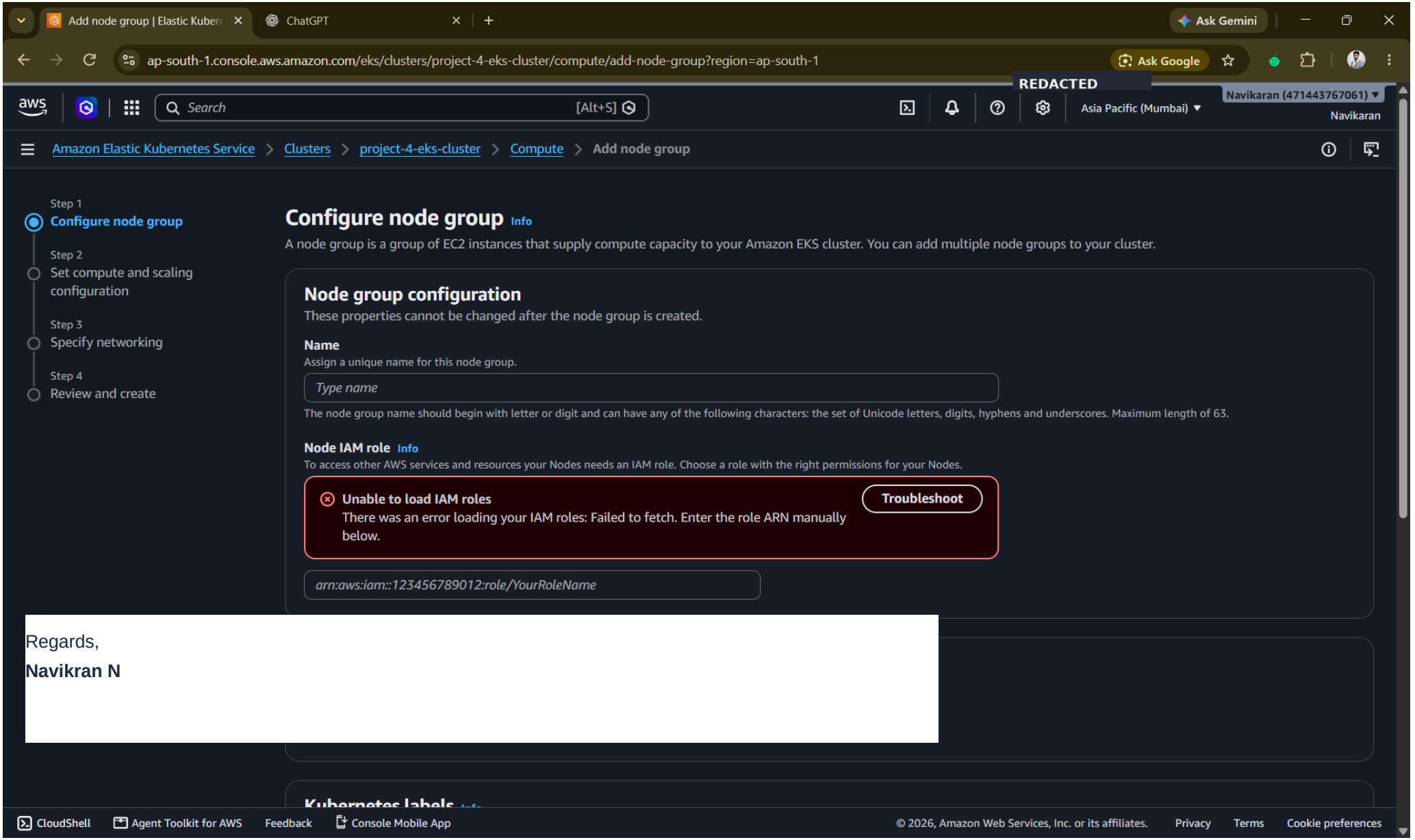


Evidence boundary: This screenshot confirms the EKS control-plane cluster is Active. It does not by itself confirm a managed node group, worker nodes, application workloads, Prometheus, or Grafana are running.

EKS Managed Node Group – IAM Role Loading Error

GUVI Project 04 | Screenshot captured 04 November 2026

When attempting to add a managed node group, the AWS console displayed: **“Unable to load IAM roles – Failed to fetch. Enter the role ARN manually below.”** This prevented the normal IAM role selection flow from proceeding in the console at that time.



Evidence boundary: This screenshot proves the IAM role list failed to load in the console during this attempt. It does not independently prove that the AWS account was formally suspended. The learner reports broader AWS account access/operational restrictions; the exact account status should be confirmed by AWS Support.

Evaluator Clarification Letter

GUVI Project 04 – Three-Tier Blog Application

Dear Evaluator,

I would like to clarify the deployment environment and the limitation I faced while completing this project.

I initially planned to deploy the application on Amazon EKS and then configure Prometheus and Grafana for monitoring. I successfully created the EKS cluster, and the AWS console showed the cluster status as Active. However, when I attempted to add the managed node group, the AWS console failed to load the available IAM roles and displayed the error “Unable to load IAM roles – Failed to fetch.”

At the same time, my personal AWS account was experiencing access and operational limitations, and I was unable to proceed reliably with the remaining AWS setup. I have contacted AWS Support regarding the account issue, but the required access was not restored in time for the project submission. I therefore could not complete the EKS worker-node setup or validate Prometheus and Grafana on EKS.

Rather than leaving the project incomplete, I used Docker, Docker Compose, Jenkins, and Kubernetes on Docker Desktop to build, test, and deploy the application. The attached execution evidence documents the application, container image builds, Jenkins pipeline, Docker Hub image publishing, SonarQube Quality Gate result, Trivy report artifacts, and successful local Kubernetes rollout.

I have included the AWS EKS Active-cluster screenshot and the IAM role loading error screenshot as supporting context. I respectfully request that the completed application and CI/CD work be considered, with the AWS EKS managed-node-group and Prometheus/Grafana portions treated as blocked by the AWS account access issue rather than represented as completed.

Thank you for your understanding.

Regards,
Project 04 Candidate

Submission note: The report distinguishes completed local implementation from AWS work that could not be completed. No live EKS workload or Prometheus/Grafana deployment is claimed.